

UNIVERSIDADE DE AVEIRO

Relatório do Projeto de Base de Dados

Sistema de Gestão NewModusApp

Engenharia de Computadores e Informática

DEPARTAMENTO DE ELETRÓNICA, TELECOMUNICAÇÕES E INFORMÁTICA

Tiago Pita

Nº Mecanográfico: 120152

Paulo Lacerda

Nº Mecanográfico: 120202

3 de junho de 2026

Conteúdo

1	Introdução	1
2	Análise de Requisitos	2
2.1	Entidades	2
2.2	Funcionalidades	2
3	Conceptualização da base de dados	3
3.1	Diagrama ER	4
3.2	Modelo Relacional	5
3.3	Normalização	5
4	Construção da Base de Dados	6
4.1	Criação das Tabelas	6
4.2	Inserção de dados	7
5	SQL Programming	8
5.1	UDF's (User-Defined Functions)	8
5.2	Views	9
5.3	Stored Procedures	10
5.4	Triggers	12
5.5	Indexes	14
6	Interface Gráfica	15
6.1	Demonstração de Funcionalidades	15
6.2	Configuração do Ambiente e Scripts de Instalação	16
6.2.1	Configuração da Aplicação (C#)	16
6.2.2	Scripts de Instalação e Sincronização (.bat)	16
7	Conclusão	17
8	Bibliografia	18

1 Introdução

No âmbito da unidade curricular de Base de Dados, propusemos a realização deste trabalho prático, cujo tema recai sobre a gestão de um atelier de roupa e confeção por medida.

A base de dados criada incide, portanto, na gestão do stock de tecidos e materiais, registo de clientes e respetivas medidas anatómicas, controlo de compras de pronto-a-vestir e vendas por encomenda.

O desenvolvimento deste trabalho prático tem como objetivo a aplicação prática dos conhecimentos adquiridos nas aulas teóricas e práticas desta unidade curricular. O sistema desenvolvido (**NewModusApp**) visa digitalizar e centralizar a informação que tradicionalmente dependia de apontamentos em papel, automatizando cálculos de lucros e facilitando o acesso rápido a descrições e customizações de produtos.

2 Análise de Requisitos

Tendo em conta o sistema proposto efetuou-se uma análise de requisitos, de modo a levantar todas as partes chaves necessárias para uma implementação eficaz, clara e sucinta.

2.1 Entidades

- **Cliente e Fornecedor** - São entidades fundamentais do sistema e caracterizam-se por dados de contacto como o nome, telefone e e-mail.
- **Tecido e Material** - Representam o stock de matérias-primas do atelier, reunindo atributos como o nome, custo unitário, quantidade em stock e o fornecedor onde se encontra.
- **Produto Pronto** - Representa as peças de roupa já confeccionadas, categorizadas por tamanho, cor, quantidade e preço anunciado.
- **Perfil de Medida** - Representa o histórico da evolução das medidas anatómicas (braço, costas, peito, etc.) associadas a um cliente.
- **Compra** - Representa o ato de adicionar um artigo de pronto-a-vestir à faturação, contendo a data da compra, o valor total e o método de pagamento.
- **Encomenda** - Representa o ato de registar um pedido de roupa por medida a um cliente, tendo as datas de registo e entrega, os detalhes de customização e o estado do pedido.
- **Ajuste** - Representa os arranjos de costura necessários, podendo estar associado a uma encomenda, e permite registar o consumo extra de tecidos ou materiais decorrentes dessa alteração.

2.2 Funcionalidades

Convém denotar que este sistema consegue realizar as seguintes operações:

- O sistema permite o registo de clientes, fornecedores, tecidos, materiais e produtos de pronto-a-vestir.
- O sistema permite o registo de vendas diretas e a gestão de encomendas de roupa por medida.
- O sistema permite alocar os materiais ou tecidos exatos à produção de cada encomenda.
- O sistema permite a gestão de stock, atualizando-o de forma rigorosa e descontando os materiais assim que entram em produção.
- O sistema também permite a visualização de um painel geral (Dashboard) com cálculos automáticos e métricas da loja.

3 Conceptualização da base de dados

Durante o desenvolvimento do projeto, o modelo inicial sofreu algumas alterações estruturais. Estas melhorias garantem que a base de dados reflete melhor as regras do negócio e evita anomalias:

- **Gestão de Stock Centralizada:** O stock de tecidos passou para `DECIMAL(10,2)` porque o tecido é cortado ao metro. Na nova tabela `Movimentos_Stock`, para evitar várias colunas a nulo, optámos por guardar apenas o ID e o tipo de item, poupando espaço e simplificando consultas.
- **Gestão Financeira:** Adicionámos o `custo_mao_obra` e o `preco_cobrado`. Isto permite guardar o custo exato dos materiais no momento da encomenda, para que o cálculo dos lucros não seja afetado se o preço do fornecedor mudar no futuro.
- **Ajustes:** Criámos as tabelas `Ajuste_Tecido` e `Ajuste_Material`. Assim, quando há um erro na confeção, o sistema desconta de forma automática o material extra gasto.
- **Apagar em Cascata:** Usámos `ON DELETE CASCADE` nalgumas tabelas dependentes (como itens de encomenda). Assim, se apagarmos uma encomenda, os seus itens são eliminados automaticamente, evitando registos perdidos.

3.1 Diagrama ER

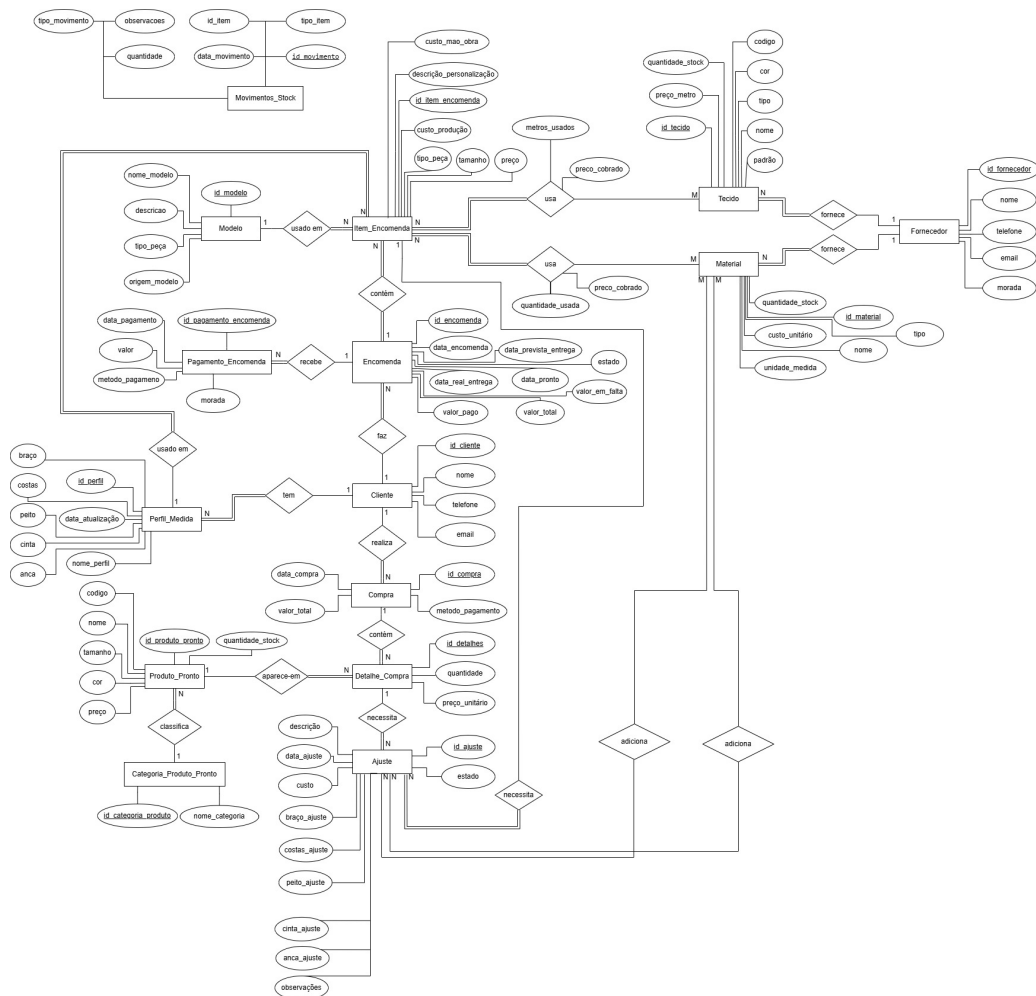


Figura 1: Diagrama Entidade-Relacionamento (DER)

3.2 Modelo Relacional

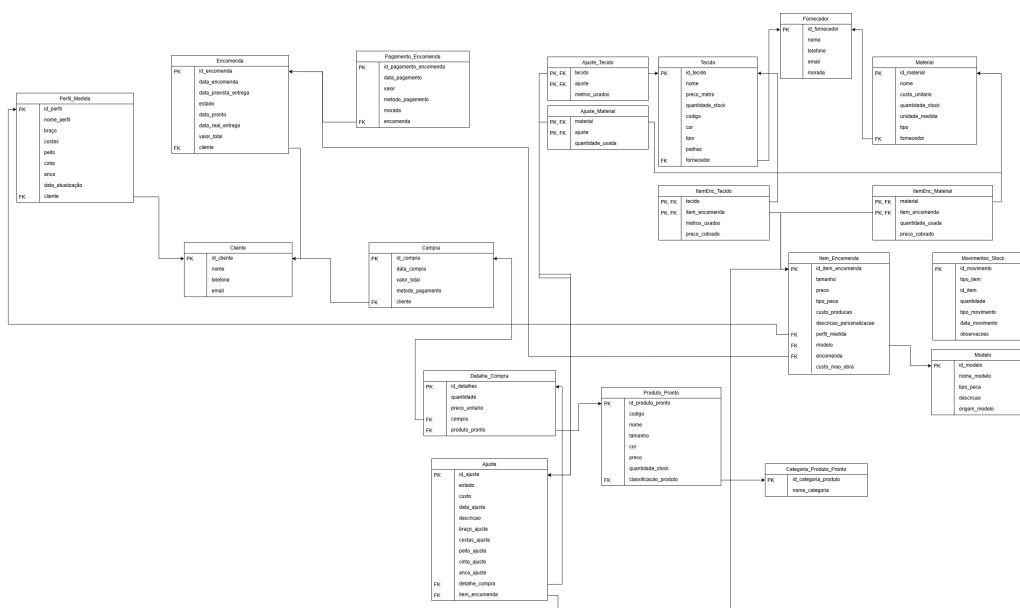


Figura 2: Modelo Relacional da Base de Dados

3.3 Normalização

Durante a criação da base de dados, garantimos que o esquema respeita a 3ª Forma Normal (3FN). Transformámos as relações de Muitos-para-Muitos (M:N) em tabelas associativas (como a `ItemEnc_Tecido`) para cumprir a 1FN. Garantimos também que todos os atributos não-chave dependem inteiramente da chave primária (2FN) e que não existem dependências transitivas (3FN). Isto evita dados repetidos e problemas ao atualizar a informação.

4 Construção da Base de Dados

Neste capítulo apresentamos a implementação da base de dados no SQL Server, abrangendo a criação das tabelas e a inserção de dados.

4.1 Criação das Tabelas

A criação das tabelas em SQL DDL incluiu a definição dos tipos de dados e das restrições (Primary Keys, Foreign Keys e restrições CHECK). Abaixo mostramos dois exemplos: a tabela de itens de encomenda e a de produtos prontos.

```
1 CREATE TABLE NM.Item_Encomenda (
2     id_item_encomenda      INT          NOT NULL ,
3     tamanho                INT          NOT NULL ,
4     preco                  DECIMAL(6,2)  NOT NULL ,
5     tipo_peca              VARCHAR(15)   NOT NULL ,
6     custo_producao         DECIMAL(10,2),
7     custo_mao_obra         DECIMAL(6,2)  NOT NULL DEFAULT 0,
8     descricao_personalizacao VARCHAR(50),
9     perfil_medida          INT          NOT NULL ,
10    modelo                  INT          NULL ,
11    encomenda               INT          NOT NULL ,
12
13    CONSTRAINT PK_Item_Encomenda PRIMARY KEY (id_item_encomenda),
14    CONSTRAINT FK_Item_Perfil_Medida FOREIGN KEY (perfil_medida)
15 REFERENCES NM.Perfil_Medida(id_perfil),
16    CONSTRAINT FK_Item_Modelo FOREIGN KEY (modelo) REFERENCES NM.Modelo(
17 id_modelo),
18    CONSTRAINT FK_Item_Encomenda FOREIGN KEY (encomenda) REFERENCES NM.
19 Encomenda(id_encomenda)
20    ON DELETE CASCADE
21 );
```

Listing 1: Criação da tabela Item_Encomenda


```

1 CREATE TABLE NM.Produto_Pronto (
2     id_produto_pronto      INT          NOT NULL,
3     codigo                 INT          NOT NULL,
4     nome                   VARCHAR(30)   NOT NULL,
5     tamanho                VARCHAR(10)   NOT NULL,
6     cor                    VARCHAR(10)   NOT NULL,
7     preco                  DECIMAL(6,2) NOT NULL,
8     quantidade_stock       INT          NOT NULL DEFAULT 0,
9     classificacao_produto  INT,
10
11     CONSTRAINT PK_Produto_Pronto PRIMARY KEY (id_produto_pronto),
12     CONSTRAINT UQ_Produto_Pronto_Codigo UNIQUE (codigo),
13     CONSTRAINT CHK_ProdutoPronto_Stock CHECK (quantidade_stock >= 0),
14     CONSTRAINT FK_Produto_Categoria FOREIGN KEY (classificacao_produto)
15 REFERENCES NM.Categoria_Produto_Pronto(id_categoria_produto)
);

```

Listing 2: Criacao da tabela Produto_Pronto

4.2 Inserção de dados

Os dados foram inseridos nas tabelas utilizando instruções DML. Adicionámos registos de teste para podermos validar o funcionamento da base de dados e testar a interface gráfica.

```

1 -- Insercao de um novo Fornecedor no sistema
2 INSERT INTO NM.Fornecedor (id_fornecedor, nome, telefone, email, morada)
3 VALUES
4 (1, 'Texteis do Ave', '253102934', 'encomendas@texteisdoave.pt', '
5   Guimaraes');
6
7 -- Insercao de um Tecido com preco e referencia ao fornecedor
8 INSERT INTO NM.Tecido (id_tecido, nome, preco_metro, quantidade_stock,
9   codigo, cor, tipo, padrao, fornecedor) VALUES
10 (1, 'La Fria 130s', 85.50, 150.00, 10001, 'Azul Navy', 'La', 'Liso', 10)
11 ;

```

Listing 3: Exemplo de instrucao INSERT para registo de Fornecedores e Tecidos.

5 SQL Programming

Para garantir a segurança e melhorar o desempenho, passámos grande parte da lógica do sistema para a base de dados. Isto protege a informação e garante que as regras do negócio são cumpridas de forma centralizada.

5.1 UDF's (User-Defined Functions)

Usámos UDF's para efetuar cálculos repetitivos. Ao manter estas funções na base de dados, evitamos duplicar código na aplicação em C#.

```
1  -- Calcula o total principal de dados associados a um cliente.
2  CREATE FUNCTION NM.fnTotalDadosAssociadosCliente
3  (
4      @id_cliente INT
5  )
6  RETURNS INT
7  AS
8  BEGIN
9      DECLARE @total INT;
10
11      SELECT @total =
12          (SELECT COUNT(*) FROM NM.Perfil_Medida WHERE cliente =
13           @id_cliente) +
14          (SELECT COUNT(*) FROM NM.Encomenda WHERE cliente = @id_cliente)
15          +
16          (SELECT COUNT(*) FROM NM.Compra WHERE cliente = @id_cliente);
17
18      RETURN ISNULL(@total, 0);
19  END;
20  GO
```

Listing 4: Funcao escalar para calcular o total de dependencias de um cliente.

```

1 -- Calcula o total de uma encomenda a partir dos itens associados.
2 CREATE FUNCTION NM.fnCalcularTotalEncomenda
3 (
4     @id_encomenda INT
5 )
6 RETURNS DECIMAL(10,2)
7 AS
8 BEGIN
9     DECLARE @total DECIMAL(10,2);
10
11     SELECT @total = SUM(preco)
12     FROM NM.Item_Encomenda
13     WHERE encomenda = @id_encomenda;
14
15     RETURN ISNULL(@total, 0);
16 END;
17 GO

```

Listing 5: Funcao escalar para obter o valor total de uma encomenda.

5.2 Views

As Views foram criadas para simplificar consultas difíceis que usam vários JOINS. Elas funcionam como uma ponte mais simples entre as tabelas reais e a aplicação.

```

1 -- Numero de encomendas em estado activo (Pendente ou Em Producao).
2 CREATE VIEW NM.vwDashboard_EncomendasAtivas
3 AS
4     SELECT COUNT(*) AS Total
5     FROM NM.Encomenda
6     WHERE estado IN ('Pendente', 'Em Producao');
7 GO

```

Listing 6: View agregadora para alimentar os cartoes do Dashboard.

```

1 -- Junta os perfis de medida ao respectivo cliente para uso na interface.
2 CREATE VIEW NM.vwPerfisMedida
3 AS
4     SELECT
5         PM.id_perfil AS ID,
6         PM.nome_perfil AS Perfil,
7         PM.cliente AS ClienteID,
8         C.nome AS Cliente,
9         PM.data_atualizacao AS Data,
10        PM.braco AS Braco,
11        PM.costas AS Costas,
12        PM.peito AS Peito,
13        PM.cinta AS Cinta,
14        PM.anca AS Anca
15    FROM NM.Perfil_Medida PM
16    INNER JOIN NM.Cliente C ON PM.cliente = C.id_cliente;
17 GO

```

Listing 7: View com JOINS para formatar dados de apresentacao na interface.

5.3 Stored Procedures

Usámos Stored Procedures para inserir, atualizar e apagar dados. Para além de serem mais rápidas, protegem a base de dados contra ataques de SQL Injection, porque obrigam à utilização de parâmetros.

```

1  -- Atualiza os campos de um material existente.
2  CREATE PROCEDURE NM.spAtualizarMaterial
3      @id_material      INT,
4      @nome              VARCHAR(30),
5      @custo_unitario    DECIMAL(6,2),
6      @quantidade_stock INT,
7      @unidade_medida    VARCHAR(15),
8      @tipo              VARCHAR(15),
9      @fornecedor        INT
10 AS
11 BEGIN
12     SET NOCOUNT ON;
13
14     UPDATE NM.Material
15     SET nome              = @nome,
16         custo_unitario    = @custo_unitario,
17         quantidade_stock  = @quantidade_stock,
18         unidade_medida    = @unidade_medida,
19         tipo              = @tipo,
20         fornecedor        = @fornecedor
21     WHERE id_material = @id_material;
22
23     IF @@ROWCOUNT = 0
24         RAISERROR('0 material indicado nao existe.', 16, 1);
25 END;
26 GO

```

Listing 8: Stored Procedure para atualizacao de dados estruturais de um material.

```

1  -- Cria uma nova encomenda e devolve o id criado.
2  CREATE PROCEDURE NM.spCriarEncomenda
3      @data_encomenda DATE,
4      @data_prevista_entrega DATE = NULL,
5      @estado NVARCHAR(15),
6      @data_pronto DATE = NULL,
7      @data_real_entrega DATE = NULL,
8      @valor_total DECIMAL(10,2),
9      @cliente INT,
10     @id_encomenda INT OUTPUT
11 AS
12 BEGIN
13     SET NOCOUNT ON;
14
15     SET @estado = NULLIF(LTRIM(RTRIM(@estado)), N'');
16
17     IF @data_encomenda IS NULL OR @estado IS NULL
18     BEGIN
19         RAISERROR('Data e estado sao obrigatorios.', 16, 1);
20         RETURN;
21     END;
22
23     SELECT @id_encomenda = ISNULL(MAX(id_encomenda), 0) + 1
24     FROM NM.Encomenda;
25
26     INSERT INTO NM.Encomenda
27         (id_encomenda, data_encomenda, data_prevista_entrega, estado,
28          data_pronto, data_real_entrega, valor_total, cliente)
29     VALUES
30         (@id_encomenda, @data_encomenda, @data_prevista_entrega, @estado,
31          @data_pronto, @data_real_entrega, @valor_total, @cliente);
32
33     SELECT @id_encomenda AS id_encomenda;
34 END;
35 GO

```

Listing 9: Stored Procedure de insercao com captura e retorno de OUTPUT param.

5.4 Triggers

Os Triggers foram essenciais para garantir regras mais complexas que as restrições normais não conseguem fazer. Eles funcionam de forma automática após modificações de dados e efetuam um *ROLLBACK* à operação caso as regras de negócio sejam violadas.

```

1  -- Bloqueio defensivo - impedir alteracoes quando nao esta pendente
2  -- Nao podemos deixar adicionar/remover tecidos se a encomenda ja
   estiver a ser feita.
3  CREATE TRIGGER NM.trgBloquearAlteracoesTecidoEmProducao
4  ON NM.ItemEnc_Tecido
5  AFTER INSERT, UPDATE, DELETE
6  AS
7  BEGIN
8      SET NOCOUNT ON;
9
10     -- Verifica se alguma das encomendas afetadas nao esta "Pendente"
11     IF EXISTS (
12         SELECT 1 FROM inserted i
13         JOIN NM.Item_Encomenda ie ON i.item_encomenda = ie.
id_item_encomenda
14         JOIN NM.Encomenda e ON ie.encomenda = e.id_encomenda
15         WHERE e.estado != 'Pendente'
16     ) OR EXISTS (
17         SELECT 1 FROM deleted d
18         JOIN NM.Item_Encomenda ie ON d.item_encomenda = ie.
id_item_encomenda
19         JOIN NM.Encomenda e ON ie.encomenda = e.id_encomenda
20         WHERE e.estado != 'Pendente'
21     )
22     BEGIN
23         RAISERROR('Nao pode alterar materiais de uma encomenda que ja
nao esta Pendente.', 16, 1);
24         ROLLBACK TRAN;
25     END
26 END;
27 GO

```

Listing 10: Trigger defensivo que bloqueia alteracoes a componentes em producao.

```

1 -- Garante regras basicas tambem quando os dados entram fora da
  aplicacao.
2 CREATE TRIGGER NM.trgPerfilMedida_ValidarValores
3 ON NM.Perfil_Medida
4 AFTER INSERT, UPDATE
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8
9     IF EXISTS (
10         SELECT 1
11         FROM inserted
12         WHERE LTRIM(RTRIM(nome_perfil)) = ''
13             OR braco < 0
14             OR costas < 0
15             OR peito < 0
16             OR cinta < 0
17             OR anca < 0
18     )
19     BEGIN
20         RAISERROR('0 perfil de medida tem valores invalidos.', 16, 1);
21         ROLLBACK TRAN;
22     END;
23 END;
24 GO

```

Listing 11: Trigger para validacao avancada de integridade de metricas inseridas.

5.5 Indexes

Para que as consultas à base de dados sejam mais rápidas, criámos alguns Indexes. Escolhemos colunas que são muito utilizadas em condições de pesquisa (*WHERE*) ou nas ligações entre as tabelas (*JOINS*).

```

1 -- Acelera pesquisas de clientes por email.
2 IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Cliente_Email
  ' AND object_id = OBJECT_ID(N'NM.Cliente'))
3     CREATE INDEX IX_Cliente_Email ON NM.Cliente(email);
4 GO

```

Listing 12: Criacao de Indice para pesquisa textual.

```

1 -- Acelera consultas de encomendas por cliente.
2 IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'
  IX_Encomenda_Cliente' AND object_id = OBJECT_ID(N'NM.Encomenda'))
3     CREATE INDEX IX_Encomenda_Cliente ON NM.Encomenda(cliente);
4 GO

```

Listing 13: Criacao de Indice em chave estrangeira para otimizacao de JOINS.

6 Interface Gráfica

Para mostrar a base de dados a funcionar, criámos uma aplicação em C# utilizando *Windows Forms*.

6.1 Demonstração de Funcionalidades

A interface gráfica permite aos utilizadores interagirem com a base de dados de forma intuitiva, sem precisarem de conhecimentos de SQL. O programa utiliza as Views e Stored Procedures que criámos anteriormente para consultar e alterar os dados de forma rápida e segura.

Abaixo apresentam-se capturas de ecrã dos módulos principais do sistema:

Clientes

Dados do Cliente

ID Cliente: 27

Nome: Beatriz Correia

Telefone: 915462738

Email: bea.correia@gmail.com

Buttons: Guardar, Editar, Apagar, Limpar

Medidas Encomendas

ID	Nome	Telefone	Email
5	Ana Beatriz Marques	967382194	anabeatrizmarques@gmail.com
20	André Teixeira	931284756	andreteira@hotmail.com
2	António Costa	935617263	antonio.costa@hotmail.com
327	Beatriz Correia	915462738	bea.correia@gmail.com
24	Bruno Alves	937481526	alves_bruno@sapo.pt
3	Carla Mendes	926371842	carla.mendes99@gmail.com
13	Catarina Fernandes	921637465	cat.fernandes9@gmail.com
25	Daniela Sousa	925638174	daniela_sousa@gmail.com
31	Diana Mota	916372845	diana_mota@gmail.com
16	Diogo Azevedo	936571824	d.azevedo@gmail.com
34	Eduardo Lima	962384715	edulima_eng@hotmail.com
32	Fábio Morais	939281746	fabiomorais11@gmail.com
17	Filipa Lourenço	927164835	filipa.lo@sapo.pt
38	Giorgio Tavares	963284715	giavanes85@yahoo.com
14	Hugo Gomes	965382714	hugogomes_jr@hotmail.com
19	Inês Castro	913745835	castro.ines@gmail.com
37	Jessica Matos	921475836	jess_matos@gmail.com
9	Joana Ribeiro	917263645	joaninha.ribeiro@yahoo.com
6	Jose Almeida	918475629	j.almeida1975@gmail.com
23	Lara Pires	918274563	lara.pires@gmail.com
10	Luís Ferreira	964512938	lferreira_arg@gmail.com
26	Marco Leite	961273845	marcoleite1@hotmail.com
1	Maria João Silva	912345678	mjosilva@gmail.com
40	Mário Brito	934571826	mariobrito_b@gmail.com
7	Marta Faria	932847162	marta_faria@hotmail.com
28	Miguel Pinho	938174825	mguelpinho@yahoo.com
18	Nuno Rodrigues	966273145	nuno.rodrigues.8@gmail.com
30	Paulo Rocha	964827135	paulorocha_cr@sapo.pt
4	Pedro Oliveira	916452738	pedrinho.oliveira@sapo.pt
29	Raquel Neves	923648175	raquelneves.neves@gmail.com
36	Ricardo Magalhães	935617284	ricardo.magalhaes@sapo.pt
8	Rui Pinto	923847161	rulpinto88@gmail.com
11	Kara Martins	615476074	m.kara_martins@hotmail.com

Figura 3: Ecrã principal de gestão e consulta de Clientes.

Pedidos Pendentes

< Pedidos pendentes

Buttons: Iniciar Producao, Marcar Concluida, Cancelar, Adicionar materiais, Ver materiais

ID	Cliente	Data	DataPrevista	Estado	DataPronta	DataEntrega	ValorTotal	TotalItems
77	Jessica Matos	28/05/2026	28/06/2026	Pendente			750.00	750.00
76	Catarina Fernandes	25/05/2026	25/06/2026	Em Produção			650.00	650.00
75	Ricardo Magalhães	22/05/2026	22/06/2026	Em Produção			550.00	550.00
74	Rui Pinto	18/05/2026	18/06/2026	Em Produção			850.00	850.00
73	Diana Mota	15/05/2026	15/06/2026	Em Produção			480.00	480.00
68	Susana Viana	12/05/2026	12/06/2026	Pronta	01/06/2026		520.00	520.00
67	Diogo Azevedo	10/05/2026	10/06/2026	Pronta	01/06/2026		450.00	450.00
66	Raquel Neves	08/05/2026	08/06/2026	Pronta	01/06/2026		750.00	750.00
65	Pedro Oliveira	05/05/2026	05/06/2026	Pronta	31/05/2026		650.00	650.00
72	Nuno Rodrigues	04/05/2026	04/06/2026	Pronta	01/06/2026		700.00	700.00
71	Beatriz Correia	02/05/2026	02/06/2026	Pronta	30/05/2026		600.00	600.00
70	Daniela Sousa	28/04/2026	28/05/2026	Pronta	22/05/2026		400.00	400.00
69	Luís Ferreira	25/04/2026	25/05/2026	Pronta	20/05/2026		850.00	850.00

Produtos do pedido selecionado

ID	Perfil	Modelo	Tamanho	Preco	TipoPeca	CustoProducao	Descricao	Materiais
75	Ricardo - Pessoal	Sobretudo de Lã	42	550.00	Casaco	278.00	Forno contraste	✓

Figura 4: Módulo de gestão de Encomendas por Medida.

6.2 Configuração do Ambiente e Scripts de Instalação

Para colocar a aplicação a funcionar num novo ambiente, é necessário ajustar a indicação do servidor tanto no código C# como nos scripts de automatização criados.

6.2.1 Configuração da Aplicação (C#)

No código-fonte, o endereço do SQL Server está centralizado no ficheiro `Data/DbConfig.cs`. Por omissão, o sistema está configurado para procurar uma instância local, sendo necessário alterar o valor da variável `Server` para o nome ou IP correto do servidor onde a base de dados foi injetada.

```
1 // Ficheiro: Data/DbConfig.cs
2
3 public static string Server = ".\SQLSERVER";
```

Listing 14: Definição do servidor no ficheiro `DbConfig.cs`.

6.2.2 Scripts de Instalação e Sincronização (.bat)

Para facilitar o processo, criámos os scripts `install_database.bat` (para criar e instalar a base de dados do zero) e `sync_database.bat` (para sincronizar tabelas e rotinas).

Estes ficheiros utilizam o utilitário `sqlcmd` e incluem uma rotina de verificação que tenta detetar automaticamente se o servidor está a correr sob o nome `.\SQLSERVER` ou `localhost`. Caso a instância do servidor remota ou local tenha um nome diferente destes dois, é necessário editar os ficheiros `.bat` e atualizar a variável `SERVER` para o nome correto antes da execução.

```
1 :: Excerto dos ficheiros install_database.bat e sync_database.bat
2 :: 1. SQLEXPRESS
3 sqlcmd -S .\SQLSERVER -E -Q "SELECT 1" >nul 2>&1
4 if %ERRORLEVEL% EQU 0 (
5     set SERVER=.\SQLSERVER
6     goto Execute
7 )
8
9 :: 2. localhost
10 sqlcmd -S localhost -E -Q "SELECT 1" >nul 2>&1
11 if %ERRORLEVEL% EQU 0 (
12     set SERVER=localhost
13     goto Execute
14 )
```

Listing 15: Rotina de verificação e definição do servidor nos ficheiros batch.

7 Conclusão

Em suma, o trabalho de desenvolvimento da base de dados para a gestão do atelier de roupa **NewModusApp** foi realizado com sucesso, aplicando os conhecimentos adquiridos na unidade curricular de Base de Dados.

A análise de requisitos permitiu identificar as entidades e funcionalidades essenciais para o sistema, como o registo das medidas dos clientes e o controlo do consumo de materiais. A construção da base de dados envolveu a criação das tabelas, seguida pela inserção de dados relevantes. Durante o processo, foram utilizadas *Stored Procedures*, *Triggers* e *UDF's* para facilitar operações complexas e automatizar a atualização do inventário.

Medidas de segurança foram implementadas para prevenir ataques de *SQL Injection*, através do uso de parâmetros nas *Procedures*. Além disso, configurámos mensagens de erro através do comando **RAISERROR**, para ajudar a aplicação e os utilizadores a corrigirem os dados inseridos.

A base de dados foi integrada a uma interface gráfica em C# para demonstrar as funcionalidades do sistema.

No geral, o trabalho cumpriu o objetivo de aplicar na prática a teoria lecionada nas aulas, sendo fundamental para compreender a importância da organização e eficiência num sistema de base de dados.

8 Bibliografia

- Slides e PDFs teóricos disponibilizados na página de *eLearning* da Unidade Curricular de Bases de Dados.
- Documentação oficial da Microsoft (T-SQL, Stored Procedures, Triggers). Disponível em: <https://docs.microsoft.com/en-us/sql/t-sql/>
- Stack Overflow (Consultas para resolução de dúvidas pontuais na manipulação de queries). Disponível em: <https://stackoverflow.com/>
- Documentação oficial da Microsoft para C# e *Windows Forms*. Disponível em: <https://docs.microsoft.com/en-us/dotnet/desktop/winforms/>